DEAC: A Defensive Ensemble Framework for Automatic Graph-Based Clustering of High-Dimensional Data

Hussein Ali Ismael[1]*

[1]*Department of Computer Science, College of Science, University of Diyala, Diyala, Iraq*

* Corresponding author's Email: Al-Shaibani@uodiyala.edu.iq

Abstract: Clustering high-dimensional data without a known cluster count conventionally returns the partition that maximizes an internal quality score such as modularity or silhouette. We show this is unsafe: across twelve benchmarks, metaheuristically optimizing internal objectives degrades external agreement, because internal and external quality are only weakly correlated. We study this misalignment directly. Using a label-aware oracle that selects the best engine per dataset as an upper bound, we measure how much any label-free rule recovers: a three-layer QualityGate recovers about two-thirds of the oracle's gain, and learned, stability, and consensus selectors never close the gap, indicating it is structural. The multi-representation Shared Nearest Neighbor and Leiden ensemble the gate arbitrates is itself a competitive label-free method that infers the cluster count, remaining statistically indistinguishable from strong classical and deep baselines while trailing only one recent method. We package these as DEAC and release all per-seed results and scripts.

Keywords: Clustering, High-dimensional data, Graph-based clustering, Differential evolution, Shared nearest neighbors, Leiden algorithm, Ensemble clustering

1. Introduction

Clustering seeks to uncover hidden structure in data without class labels . As samples and feature dimensions both grow, clustering high-dimensional data has grown harder , and three concrete difficulties follow from that growth:

1. **Unknown cluster count.** Most methods (KMeans, Spectral, GMM) require the number of clusters C as an input, which is rarely available in practice.

2. **Representation dependence.** Clustering quality depends heavily on the choice of dimensionality reduction—PCA, UMAP, or t-SNE—and no single choice is best across all datasets.

3. **Hyperparameter sensitivity.** Graph-based clustering involves four tightly coupled hyperparameters—k (neighbors), d' (dimensions), q (pruning), r (resolution)—whose best values differ markedly from one dataset to the next.

Graph-based clustering addresses the first difficulty but inherits the other two. Methods in this family build a similarity graph and apply community detection to recover clusters of arbitrary shape with C determined automatically. Their quality, however, still depends on the graph hyperparameters and on the representation used—gaps that an adaptive framework can close.

Contributions. Building on that observation, we propose DEAC (Defensive Ensemble Adaptive Clustering). This paper makes four contributions:

1. **An internal–external misalignment result (primary).** On twelve benchmarks, metaheuristically optimizing internal objectives *reduces* external accuracy: our DE-optimized engine, despite maximizing modularity and silhouette, scores *below* a fixed baseline in mean ARI (0.563 vs. 0.580), and silhouette barely tracks ARI (rank correlation ≈ 0.08). This reframes "which partition scores best internally" as the wrong question.

2. **A quantified oracle–deployable gap.** We define a label-aware oracle (mean ARI 0.646) as an upper bound and measure how far label-free selection reaches: a three-layer QualityGate recovers 65% of its gain over the baseline, and across learned, stability, and consensus selectors the best is 64%. The gap resists every standard label-free approach, indicating it is structural rather than a tuning artifact.

3. **A competitive label-free, cluster-count-inferring ensemble (EMR-CGC).** A multi-representation SNN–Leiden ensemble (mean ARI 0.631) that is statistically on par with strong classical and deep baselines *without* being given the cluster count—showing that the limitation is recognition, not clustering power. A complementary DE engine (AMR-DE) covers specific regimes.

4. **Full reproducibility artifacts.** We release the per-seed results (internal scores, cluster counts, gate decisions) as a full-precision matrix, with every figure/table/test script, all generated from a single canonical run, and evaluate with Wilcoxon, Friedman, and Nemenyi tests against five classical and four deep baselines.

The rest of the paper follows this order. Section 2 surveys related work. Section 3 details the DEAC framework. Section 4 reports the experiments. Section 5 discusses findings and limits; Section 6 concludes.

2. Related Work

We group prior work into six strands and close with DEAC's position in that space.

Graph-Based Clustering

Graph-based methods build a similarity graph and apply community detection. Louvain maximizes modularity greedily but may return disconnected communities. Leiden fixes that with a refinement phase that guarantees connectivity. Follow-up work has refined KNN-graph construction and sparsified edges through pruning . Whichever detector is used, the input graph depends on a prior dimensionality reduction.

Dimensionality Reduction

PCA is linear and misses manifold structure. UMAP preserves both local and global structure and is generally stronger than t-SNE for downstream clustering . Its sensitivity to hyperparameters, however, is well documented—which again favors adaptive selection over fixed defaults. When adaptation is framed as optimization, metaheuristics become relevant.

Metaheuristic Optimization for Clustering

Differential Evolution (DE) has been used to tune clustering objectives . Grey Wolf Optimizer with Lévy flights offers more aggressive exploration, and recent hybrids combine several heuristics—though they typically operate on a single representation. Representation diversity, which we pursue later, is rarely the primary target.

SNN and DPC Approaches

The Shared Nearest Neighbor (SNN) score sets edge weight from common neighbors rather than raw distance, which yields a more stable similarity. DPC-AKNN pairs adaptive KNN with geodesic distance and reports ARI$>$ 0.95 on small manifold datasets (≤ 366 samples), but still requires the true C as input—a reliance it shares with the deep methods surveyed next.

Deep Clustering

A parallel line of work learns the representation and the clustering jointly. DEC self-trains an autoencoder embedding with a KL objective; IDEC adds a reconstruction term, and DCN couples the autoencoder with k-means. N2D instead clusters the UMAP manifold of an autoencoded embedding and is a strong, lightweight baseline. All of these, however, require the cluster count C as input—the very assumption DEAC removes—which motivates our direct empirical comparison against them in §4.

Ensemble Clustering

Ensembles combine multiple partitions through co-association matrices or consensus voting . Most ensembles draw from a single representation, so partition diversity is bounded by that single view. These six strands together inform how DEAC is positioned.

Positioning of DEAC

Table 1 compares the features that distinguish graph-clustering pipelines. DEAC is the only method that combines adaptive representation (AR), automatic k (AK), hyperparameter optimization (HO), ensembling (EN), and a quality gate (QG).

Table 1. *Feature comparison with related methods.*

Method	AR	AK	HO	EN	QG
KMeans	–	–	–	–	–
Spectral Clustering	–	–	–	–	–
HDBSCAN	–	✓	–	–	–
Louvain	–	✓	–	–	–
Leiden	–	✓	–	–	–
DEC	–	–	✓	–	–
DPC-AKNN	–	–	partial	–	–
Cluster Ensembles	–	–	–	✓	–
DEAC (ours)	✓	✓	✓	✓	✓

3. Proposed Method: DEAC Framework

System Overview

The DEAC framework is organized as four sequential layers (Fig. 1): an *Input Layer* receiving raw data, a *Preprocessing Layer* that caches multiple representations, an *Engine Layer* where three clustering engines run in parallel, and an *Arbitration Layer* in which the QualityGate selects the final output. Before examining each engine in turn, we describe the graph-clustering pipeline they share.

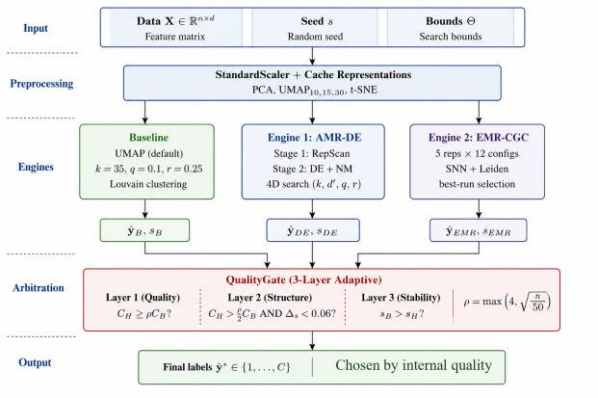
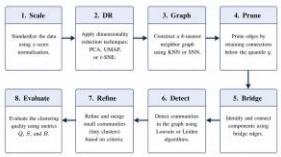


Figure 1. *DEAC framework architecture. Four processing layers with three parallel clustering engines whose outputs are arbitrated by the QualityGate.*

Graph-Based Clustering Pipeline

Both engines drive the same eight-step pipeline (Fig. 2) that turns raw data $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ into labels $\hat{\boldsymbol{y}}$. The engines differ only in how they score and search inside that pipeline, starting with AMR-DE.



$$w_{ij}^{KNN} = (1 + \cos(\boldsymbol{z}_i, \boldsymbol{z}_j))/2 \; w_{ij}^{SNN} = |N_k(i) \cap N_k(j)|/k \; Q = \frac{1}{2m}\sum_{ij}\left[A_{ij} - \frac{d_i d_j}{2m}\right]\delta(c_i, c_j) \; S = \frac{1}{n}\sum_i \frac{b(i)-a(i)}{\max\{a(i),b(i)\}} \; B = H(\hat{\boldsymbol{y}})/\log C$$

Figure 2. *Graph-based clustering pipeline (8 steps) shared by both engines. Key equations for KNN/SNN weighting and quality metrics are shown below.*

Engine 1: AMR-DE

AMR-DE (Fig. 3) runs in two stages. **Stage 1 (RepScan)** scores five representations with default graph parameters and selects the one that maximizes the composite score

$$s_{\text{rep}} = 0.30\,\text{MOD}_{01} + 0.40\,\text{SIL}_{01} + 0.30\,\text{CH}_{01},$$

where MOD_{01}, SIL_{01}, and CH_{01} are the [0,1]-normalized modularity, silhouette, and Calinski–Harabasz scores. **Stage 2 (Multi-Restart DE)** then optimizes four graph hyperparameters $\boldsymbol{\theta} = (k, d', q, r)$ on the fixed representation $\boldsymbol{Z}^*$. The fitness is a weighted sum of five quality metrics with penalties:

$$f(\boldsymbol{\theta}) = -\left(\sum_{m \in \mathcal{M}} w_m \cdot m_{01} - \text{penalties}\right),$$

with $w_{\text{MOD}} = 0.30$, $w_{\text{SIL}} = 0.25$, $w_{\text{CH}} = 0.20$, $w_{\text{DB}} = 0.15$, $w_{\text{BAL}} = 0.10$. AMR-DE commits to one representation; the second engine, by contrast, aggregates many.

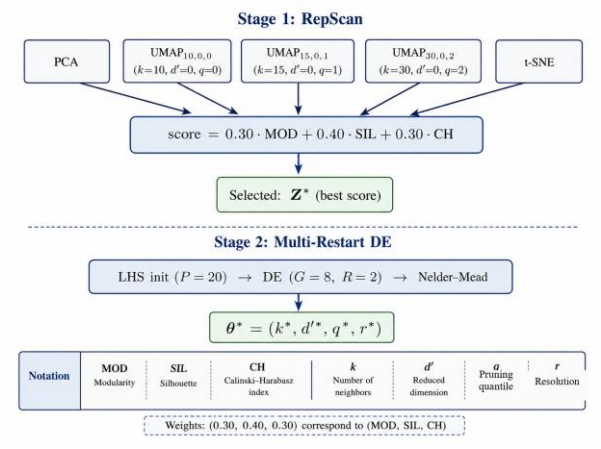


Figure 3. *Engine 1 (AMR-DE): Stage 1 selects the best representation via RepScan; Stage 2 optimizes graph hyperparameters via multi-restart DE with Nelder–Mead refinement.*

Engine 2: EMR-CGC

EMR-CGC (Fig. 4) produces 60 base clusterings—five representations × $|K| = 4$ values of k × $|R| = 3$ resolutions—using Shared Nearest Neighbor (SNN) graphs and Leiden community detection. The SNN edge weight

$$w_{ij}^{\text{SNN}} = \frac{|N_k(i) \cap N_k(j)|}{k}$$

captures structural similarity through shared neighbors and is less affected by noise and varying density than raw distance. The best base clustering is chosen by

$$s = 0.35\,\text{MOD}_{01} + 0.40\,\text{SIL}_{01} + 0.25\,\text{BAL}_{01}.$$

With two independent candidates in hand, arbitration becomes the next question.

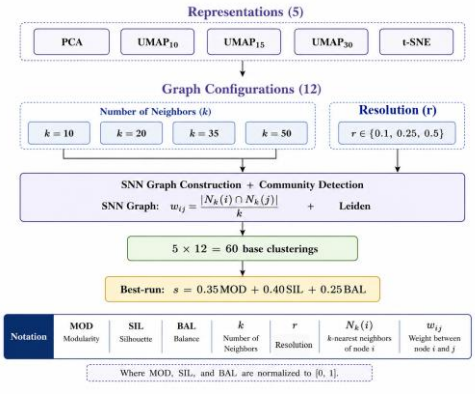


Figure 4. *Engine 2 (EMR-CGC): Ensemble of 60 base clusterings from 5 representations × 12 graph configurations, using SNN graphs with Leiden community detection. Best run is selected by quality score.*

QualityGate

The QualityGate (Fig. 5) is a three-layer adaptive filter that selects $\max(\hat{\boldsymbol{y}}_B, \hat{\boldsymbol{y}}_{DE}, \hat{\boldsymbol{y}}_{EMR})$ using internal quality scores. In Fig. 5, B denotes the baseline partition $\hat{\boldsymbol{y}}_B$ and H (the challenger, labelled "hybrid" there) is whichever engine output $\hat{\boldsymbol{y}}_{DE}$ or $\hat{\boldsymbol{y}}_{EMR}$ scores higher internally; the gate accepts H only when it clears all three layers and otherwise falls back to B. Its C-ratio threshold $\rho = \max(4, \sqrt{n/50})$ is dataset-adaptive: larger datasets are permitted proportionally more clusters, while small ones are protected from over-fragmentation. Algorithms [alg:deac]–[alg:emrcgc] give the full procedure.

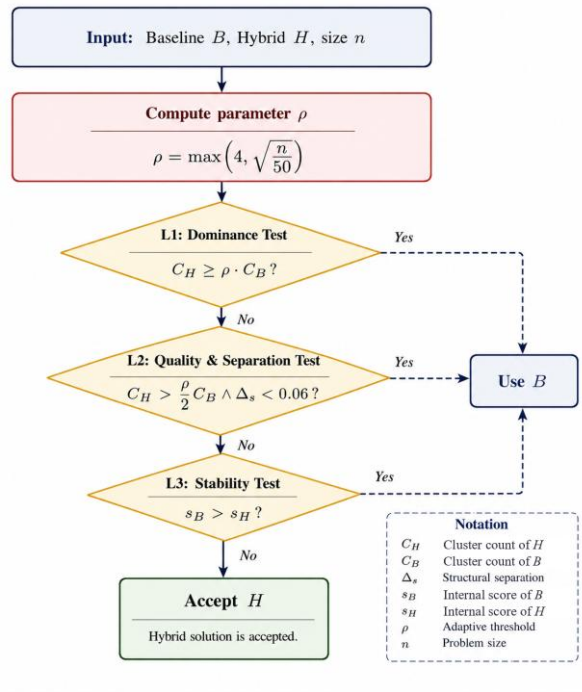


Figure 5. *QualityGate: three-layer adaptive protection. Layer 1 catches extreme over-fragmentation; Layer 2 catches moderate fragmentation with weak improvement; Layer 3 catches internal quality regression.*

Design Rationale

Two design choices warrant justification: the form of the threshold ρ and the use of SNN rather than KNN edges.

Why $\rho = \max(4, \sqrt{n/50})$. Layer 1 must reject a partition only when its cluster count is implausibly large relative to the baseline. The number of clusters that can be *statistically resolved* in a sample grows sublinearly with n: a classical rule of thumb places the maximum at $k_{\max} \approx \sqrt{n/2}$. We therefore let the tolerated cluster-count ratio grow as $\sqrt{n}$ as well, so the gate's permissiveness tracks the rate at which genuine structure can appear. Writing the threshold as $\rho = \sqrt{n/c}$, the constant $c = 50$ fixes the crossover with the floor at $n = 50 \cdot 4^2 = 800$: below ~ 800 samples the constant floor $\rho = 4$ governs, above it the $\sqrt{n}$ term does (Table 2). The floor of 4 is deliberate—baselines can severely *under*-segment (e.g. Olivetti baseline $C \approx 5$ vs. 40 true classes), so the gate must admit a several-fold increase to capture real fine structure, while still blocking the runaway fragmentation ($C_H \gg C_B$) that internal-metric optimization tends to produce. Layers 2–3 then catch the milder failure modes: moderate fragmentation paired with a negligible internal-score gain ($\Delta s < 0.06$), and outright internal-quality regression. The thresholds are intentionally conservative; because the gate can only *substitute the baseline*, an overly strict ρ costs at most the engine's gain on a dataset, whereas an overly loose ρ risks accepting a fragmented partition—an asymmetry that motivates erring toward the baseline. In the deployable implementation evaluated throughout, the gate reduces to selecting, among ρ-eligible engines, the one with the highest internal score (0.40 MOD $+ 0.35$ SIL $+ 0.25$ BAL); the Δs and regression checks are conservative safeguards that rarely bind on these datasets (§5), and the exact score normalizations, search bounds, integer rounding, graph bridging, and small-community merging are specified in the released reference implementation.

Table 2. *Size-adaptive over-fragmentation threshold $\rho = \max(4, \sqrt{n/50})$ per dataset. Below the floor/$\sqrt{n}$ crossover ($n = 800$) the constant floor governs; above it the $\sqrt{n}$ term governs.*

Dataset	n	$\sqrt{n/50}$	ρ (regime)
Wine	178	1.89	4.00 (floor)
Glass	214	2.07	4.00 (floor)
Olivetti	400	2.83	4.00 (floor)
CNAE-9	1080	4.65	4.65 ($\sqrt{n}$)
COIL-20	1440	5.37	5.37 ($\sqrt{n}$)
Optdigits	1797	5.99	5.99 ($\sqrt{n}$)
MFeat	2000	6.32	6.32 ($\sqrt{n}$)
Segmentation	2310	6.80	6.80 ($\sqrt{n}$)
ISOLET	6006	10.96	10.96 ($\sqrt{n}$)
Satimage	6430	11.34	11.34 ($\sqrt{n}$)
USPS	9298	13.64	13.64 ($\sqrt{n}$)
PenDigits	10992	14.83	14.83 ($\sqrt{n}$)

Are these constants fitted to the benchmarks? A reasonable worry is that the gate's constants—the floor of 4, the scaling $c = 50$, and the score weights—were tuned on the very datasets we report. They were not, and Table 3 shows why they need not be. Sweeping the rho floor (2–6) and c (25–150) leaves mean ARI essentially unchanged (within 0.007), and over the full weight simplex it stays in $[0.571, 0.627]$ with the default within 0.003 of the best attainable; only a degenerate modularity-only weighting fails. More tellingly, *tuning* the weights on a six-dataset development split and

evaluating on the disjoint six recovers only 41% of the oracle's gain—*below* the 65% of the fixed default. Fitting the constants to data therefore generalizes worse than leaving them fixed, so the defaults carry no label-dependent advantage.

Table 3. *QualityGate sensitivity (12 datasets, 5 seeds). Mean ARI as each constant varies around its default (floor 4, $c = 50$, weights .40/.35/.25); the gate is near-constant across the rho-guard ranges and insensitive to the score weights (only a degenerate modularity-only weighting fails). Tuning the weights on a 6-dataset development split and evaluating on the disjoint six recovers 41% of the oracle's gain versus 65% for the fixed default (-24 points), so the constants are not fitted to these benchmarks.*

Varied constant	Range tested	Mean ARI
rho floor	2–6	0.617–0.624
scaling c	25–150	0.624–0.624
score weights	full simplex	0.571–0.627
Default (floor 4, $c = 50$)	—	**0.624**

Why SNN over KNN. In high dimension, pairwise Euclidean distances concentrate—the contrast between nearest and farthest neighbors vanishes—so raw-distance edge weights become unreliable . The SNN weight $w_{ij} = |N_k(i) \cap N_k(j)|/k$ depends only on the *overlap of neighbor sets*, a rank-based quantity that is far more stable than absolute distance under this concentration, and shared-neighbor similarities are known to mitigate the curse of dimensionality directly . This motivates SNN for the high-dimensional representations in EMR-CGC's ensemble; consistent with the theory, its measured benefit is concentrated on the highest-dimensional datasets, while it is neutral-to-negative elsewhere (§5).

Algorithms

The main DEAC algorithm and its two engines are presented in Algorithms [alg:deac]–[alg:emrcgc].

Data $\boldsymbol{X} \in \mathbb{R}^{n \times d}$, seed s Cluster labels $\hat{\boldsymbol{y}}^*$ $\hat{\boldsymbol{y}}_B \leftarrow$ GraphCluster($\boldsymbol{X}$,UMAP, $k = 35, q = 0.1, r = 0.25$) $(\boldsymbol{\theta}^*, \text{rep}^*) \leftarrow$ AMR-DE$(\boldsymbol{X}, s)$ $\hat{\boldsymbol{y}}_{DE} \leftarrow$ GraphCluster$(\boldsymbol{X}, \text{rep}^*, \boldsymbol{\theta}^*)$ $\hat{\boldsymbol{y}}_{EMR} \leftarrow$ EMR-CGC$(\boldsymbol{X}, s)$ $\hat{\boldsymbol{y}}^* \leftarrow$ QualityGate$(\hat{\boldsymbol{y}}_B, \hat{\boldsymbol{y}}_{DE}, \hat{\boldsymbol{y}}_{EMR}, n)$ $\hat{\boldsymbol{y}}^*$

Data $\boldsymbol{X}$, seed s $(\boldsymbol{\theta}^*, \text{rep}^*)$ **Stage 1: RepScan** $\boldsymbol{Z}_r \leftarrow$ ComputeRep($\boldsymbol{X}$,rep, s) $\text{Score}_r \leftarrow 0.30\text{MOD} + 0.40\text{SIL} + 0.30\text{CH}$ $\text{rep}^* \leftarrow \text{argmax Score}_r$; $\boldsymbol{Z}^* \leftarrow \boldsymbol{Z}_{rep^*}$ **Stage 2: Multi-Restart DE on** $\boldsymbol{Z}^*$ Init P via Latin Hypercube Sampling Run DE for G generations Refine best with Nelder–Mead argmin fitness over restarts; rep^*

Data $\boldsymbol{X}$, seed s Cluster labels $\hat{\boldsymbol{y}}$ Reps $\leftarrow$ {PCA, $\text{UMAP}_{10,15,30}$, t-SNE} $K \leftarrow \{10,20,35,50\}$; $R \leftarrow \{0.1,0.25,0.5\}$ results $\leftarrow \emptyset$ $\boldsymbol{Z} \leftarrow$ ComputeRep($\boldsymbol{X}$,rep, s) Build SNN graph with $w_{ij} = |N_k(i) \cap N_k(j)|/k$ $\hat{\boldsymbol{y}}_i \leftarrow$ Leiden(G_{SNN}, r) $s_i \leftarrow 0.35\text{MOD} + 0.40\text{SIL} + 0.25\text{BAL}$ results $\leftarrow$ results $\cup$ $\{(\hat{\boldsymbol{y}}_i, s_i)\}$ $\text{argmax}_i s_i$

4. Experimental Evaluation

Datasets

DEAC is evaluated on 12 public benchmarks that span a wide range of n and d (Table 4). These datasets also determine the protocol below.

Table 4. *Benchmark datasets.*

Dataset	n	d	C_{true}	Domain
Wine	178	13	3	Chemistry
Glass	214	9	6	Material
Olivetti	400	4096	40	Faces
CNAE-9	1080	856	9	Text
COIL-20	1440	1024	20	Images
Optdigits	1797	64	10	Digits
MFeat	2000	649	10	Digits
Segmentation	2310	19	7	Images
ISOLET	6006	617	26	Speech
Satimage	6430	36	6	Remote Sensing
USPS	9298	256	10	Digits
PenDigits	10992	16	10	Handwriting

Evaluation Protocol

Three external validity indices are reported: Adjusted Rand Index (ARI) , Normalized Mutual Information (NMI) , and Clustering Accuracy (ACC) . Unless noted, results are averaged over five fixed seeds (1001–1005) for reproducibility. DEAC is configured with AMR-DE ($G = 8$ generations, population $P = 20$, $R = 2$ restarts over the four hyperparameters k, d', q, r) and EMR-CGC (60 base clusterings). The baseline is UMAP+Louvain with community-standard settings ($k = 35$, $q = 0.1$, resolution 0.25)—a legitimate reference rather than a deliberately weakened one; indeed it is the single strongest engine on 1 of 12 datasets. With the protocol fixed, we turn to the headline results.

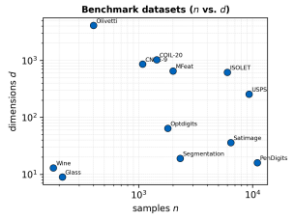
Main Results

Table 5 reports the Baseline, the three engines, the deployable **DEAC** gate, and—for reference only—the label-aware oracle. The oracle (best engine per dataset, chosen with ground-truth labels) lifts mean ARI from 0.580 to 0.646, but it is unavailable at deployment. The label-free gate recovers 65% of that gain (0.623); the ensemble alone (EMR-CGC, 0.631) recovers 77%, so the gate's arbitration slightly *lowers* mean ARI—a trade

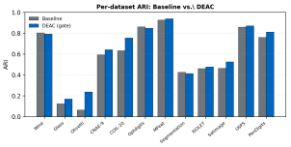
we examine in §5. Fig. 6 shows the per-dataset picture and the gate's activation. The central question is how much of the oracle a label-free rule can recover; we first place DEAC against established competitors.

Table 5. *Main results: mean ARI over 5 seeds (1001–1005), full data.* ***DEAC*** *is the deployable label-free QualityGate; Oracle is the label-selected best engine per dataset (a diagnostic upper bound, not a deployable result). Bold = best deployable value per row.*

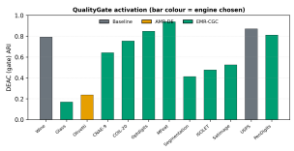
Dataset	Baseline	AMR-DE	EMR-CGC	**DEAC**	*Oracle*
Wine	**0.802**	0.774	0.797	0.791	*0.802*
Glass	0.123	0.165	0.165	**0.169**	*0.172*
Olivetti	0.063	0.237	**0.337**	0.237	*0.337*
CNAE-9	0.593	0.584	0.638	**0.642**	*0.642*
COIL-20	0.631	**0.788**	0.754	0.754	*0.789*
Optdigits	**0.861**	0.845	0.848	0.848	*0.861*
MFeat	0.926	0.903	**0.939**	**0.939**	*0.939*
Segment.	**0.425**	0.344	0.387	0.413	*0.437*
ISOLET	0.460	**0.514**	0.476	0.476	*0.516*
Satimage	0.462	0.318	0.516	**0.525**	*0.541*
USPS	0.858	0.669	**0.900**	0.871	*0.900*
PenDigits	0.759	0.617	**0.819**	0.811	*0.819*
Mean	0.580	0.563	0.631	**0.623**	*0.646*



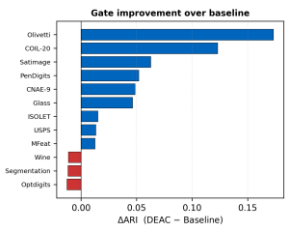
(a) Dataset characteristics (n, d)



(b) ARI comparison: Baseline vs. DEAC



(c) QualityGate activation per dataset



(d) Δ heatmap: DEAC − Baseline (sorted by ΔARI)

Figure 6. *Main experimental results. (a) Size and dimensionality of the 12 benchmark datasets. (b) Per-dataset ARI of Baseline versus DEAC. (c) Per-dataset QualityGate activation. (d) Multi-metric improvement heatmap, with datasets sorted by ΔARI descending; blue cells indicate DEAC gains over the Baseline.*

Comparison with Traditional Benchmarks

Table 6 pits DEAC against five widely used clustering algorithms. For a fair comparison, no method is given the ground-truth cluster count: each classical baseline selects its own C by silhouette analysis over a label-free grid (on the same UMAP representation), exactly as DEAC infers C through its QualityGate. This protocol is stricter than handing the baselines a known or DEAC-supplied C^*, and it exposes how much each method's quality depends on recovering C unaided.

Table 6. *ARI vs. classical methods with automatic cluster-count selection (no method is given C). Mean over 5 seeds; DEAC is the deployable gate. Bold = best per row.*

Dataset	**DEAC**	KMeans	Spectral	Agglomerative	GMM	HDBSCAN
Wine	0.791	0.788	0.788	**0.797**	0.791	0.743
Glass	**0.169**	0.135	0.119	0.133	0.134	0.129
Olivetti	**0.237**	0.011	0.009	0.011	0.011	0.037
CNAE-9	**0.642**	0.533	0.292	0.509	0.548	0.618
COIL-20	**0.754**	0.435	0.179	0.435	0.443	0.516
Optdigits	0.848	0.846	0.436	**0.855**	0.852	0.778
MFeat	**0.939**	0.937	0.448	0.937	0.937	0.925
Segment.	0.413	**0.427**	0.143	0.413	0.423	0.320
ISOLET	**0.476**	0.283	0.168	0.283	0.365	0.250
Satimage	0.525	**0.628**	0.480	0.620	0.611	0.358
USPS	**0.871**	0.639	0.470	0.639	0.639	0.705
PenDigits	0.811	0.801	0.103	**0.814**	0.800	0.800
Mean	**0.623**	0.539	0.303	0.537	0.546	0.515

The deployable DEAC gate (mean ARI 0.623) is the best method on 7 of 12 datasets and exceeds the strongest classical baseline (GMM, 0.546) by 7.7 points. Much of that margin comes from cases where silhouette-based C selection fails outright—on Olivetti it collapses every classical baseline to $C = 2$ (ARI ≈ 0.01) against the 40 true classes, whereas DEAC infers the count. Whether the aggregate advantage is statistically reliable is tested in §5; the sharper comparison is against modern deep clustering.

Comparison with Deep Clustering

Table 7 compares DEAC against four representative deep-clustering methods: N2D , DEC , IDEC , and DCN . To give the deep baselines every advantage, each is supplied with the *true* cluster count (oracle C) and trained

on the standardized raw features; DEAC alone must still discover C. All four share a fully-connected autoencoder (d–500–500–2000–10) pretrained for 100 epochs, following the original protocols. These are standard, untuned implementations: the weaker performers (DEC, IDEC), which self-train a KL objective, may trail architecture-specialized or per-dataset-tuned variants, so their scores should be read as a reasonable lower bound rather than each method's ceiling. The deployable DEAC gate reaches mean ARI 0.623 and wins on 5 of 12 datasets. It *trails* N2D (0.642), which—like DEAC—builds on UMAP and is the strongest competitor, winning on the manifold-structured datasets Olivetti, COIL-20, Satimage, and USPS. The embedded-clustering methods DEC, IDEC, and DCN (mean ARI 0.481, 0.481, 0.470) are strong only on small, low-dimensional data and degrade on high-dimensional inputs, where DEAC stays stable. Even the label-aware *oracle* (0.646) only matches N2D—so no selection rule over DEAC's engines beats the strongest deep baseline. The honest headline is competitive, count-free performance, not dominance.

Table 7. *ARI vs. deep-clustering methods (each given the true C; DEAC infers it). Mean over 5 seeds; DEAC is the deployable gate. Bold = best per row.*

Dataset	**DEAC**	N2D	DEC	IDEC	DCN
Wine	0.791	0.798	**0.870**	0.867	0.732
Glass	0.169	0.169	**0.199**	0.197	0.171
Olivetti	0.237	**0.503**	0.461	0.461	0.466
CNAE-9	**0.642**	0.468	0.069	0.069	0.019
COIL-20	0.754	**0.784**	0.504	0.505	0.510
Optdigits	**0.848**	0.812	0.516	0.517	0.547
MFeat	**0.939**	0.921	0.701	0.703	0.769
Segment.	**0.413**	0.387	0.408	0.411	0.408
ISOLET	0.476	**0.496**	0.382	0.385	0.400
Satimage	0.525	**0.658**	0.550	0.550	0.544
USPS	0.871	**0.930**	0.561	0.562	0.508
PenDigits	**0.811**	0.778	0.551	0.551	0.561
Mean	0.623	**0.642**	0.481	0.481	0.470

A dataset-level statistical analysis (Table 13) makes the picture precise, and it is sobering. After Holm–Bonferroni correction over the nine comparisons, the deployable gate significantly exceeds only the two weakest baselines (Spectral and HDBSCAN, $p_{\text{Holm}} = 0.004$). It is statistically *indistinguishable* from KMeans, GMM, Agglomerative, DEC, IDEC, and DCN, and it trails N2D. We therefore claim competitive—not superior—performance; the contribution of this paper is diagnostic (the internal–external misalignment and the oracle–deployable gap), not a new state of the art. These are all established baselines, so two very recent methods test whether the same standing holds against the current state of the art.

Comparison with Recent Methods

We additionally compare against two recent clustering methods: DPC-AKNN , a 2026 density-peak method with adaptive KNN and geodesic distance, and PRCut , a 2025 probabilistic ratio-cut deep method. Both are given the true cluster count; PRCut is run on the *same* UMAP representation as our engines (a controlled common-representation comparison) using the authors' released implementation, while DPC-AKNN uses its native raw-feature geodesic pipeline. Table 8 shows the deployable gate (0.623) and EMR-CGC (0.631) are on par with PRCut (0.631) and close to N2D (0.642), and well above DPC-AKNN (0.420)—while, unlike all three comparators, inferring the cluster count rather than being told it. Being competitive with the latest methods *without* the count, not beating them, is the honest claim. Whether this standing survives on metrics beyond ARI is the next check.

Table 8. *Comparison with recent methods (mean ARI over 12 datasets). DPC-AKNN and PRCut, like the deep baselines, are given the true cluster count; our gate and EMR-CGC infer it. PRCut is run on the same UMAP representation (common-representation comparison); DPC-AKNN uses its native raw-feature geodesic pipeline (stratified cap 2000, its $O(n^3)$ cost). Our label-free method is on par with PRCut and N2D and well above DPC-AKNN, without being told C.*

Method	Representation	Given C?	Mean ARI
DEAC gate (ours)	UMAP+SNN ensemble	no (infers C)	**0.623**
EMR-CGC (ours)	UMAP+SNN ensemble	no (infers C)	0.631
PRCut (2025)	common UMAP	yes	0.631
DPC-AKNN (2026)	native (raw+geodesic)	yes	0.420
N2D (deep)	autoencoder+UMAP	yes	0.642
Oracle (upper bound)	—	uses labels	*0.646*

Multi-Metric Comparison

Table 9 shows the ARI picture holds across NMI and ACC: the deployable gate is competitive with the strong classical baselines on all three external indices without dominating them. The per-dataset breakdown for NMI

and ACC appears in Tables 10–11, and the same values are plotted dataset-by-dataset in Fig. 7; Fig. 8 collapses the indices onto one radar plot.

Table 9. *Mean ARI/NMI/ACC across 12 datasets (5 seeds). DEAC is the deployable gate; the oracle row is a diagnostic upper bound.*

Method	Mean ARI	Mean NMI	Mean ACC
Baseline (UMAP)	0.580	0.720	0.675
AMR-DE	0.563	0.727	0.623
EMR-CGC	0.631	0.760	0.708
DEAC (gate)	**0.623**	**0.753**	**0.699**
Oracle (upper bnd)	0.646	0.762	0.720
KMeans	0.539	0.691	0.633
Spectral	0.303	0.537	0.454
Agglomerative	0.537	0.692	0.635
GMM	0.546	0.691	0.636
HDBSCAN	0.515	0.696	0.613

Table 10. *Per-dataset NMI (mean over 5 seeds). DEAC is the deployable gate.*

Dataset	Baseline	AMR-DE	EMR-CGC	**DEAC**
Wine	**0.795**	0.774	0.791	0.787
Glass	0.245	**0.359**	0.352	0.355
Olivetti	0.421	0.648	**0.727**	0.648
CNAE-9	0.716	0.698	0.718	**0.722**
COIL-20	0.904	0.908	**0.919**	**0.919**
Optdigits	**0.889**	0.883	0.885	0.885
MFeat	0.930	0.916	**0.938**	**0.938**
Segment.	**0.628**	0.604	0.624	0.626
ISOLET	0.758	0.754	**0.768**	0.766
Satimage	0.614	0.556	0.631	**0.638**
USPS	0.879	0.808	**0.894**	0.884
PenDigits	0.857	0.812	**0.878**	0.876

Table 11. *Per-dataset ACC (mean over 5 seeds). DEAC is the deployable gate.*

Dataset	Baseline	AMR-DE	EMR-CGC	**DEAC**
Wine	**0.933**	0.921	0.930	0.928
Glass	**0.489**	0.336	0.334	0.336
Olivetti	0.123	0.311	**0.447**	0.311
CNAE-9	0.752	0.670	**0.779**	0.777
COIL-20	0.752	**0.833**	0.790	0.790
Optdigits	0.897	0.870	0.906	**0.907**
MFeat	0.948	0.932	**0.972**	**0.972**
Segment.	**0.489**	0.376	0.434	0.474
ISOLET	0.528	**0.553**	0.545	0.544
Satimage	0.541	0.391	0.614	**0.626**
USPS	0.909	0.702	**0.937**	0.920
PenDigits	0.735	0.586	**0.808**	0.798

External indices say nothing about how well each method recovers the *number* of clusters. Table 12 reports predicted counts against C_{true}, and here the deployable method is weak: the gate's mean absolute count error is large and it matches C_{true} on only 2 of 12 datasets. Recovering the right granularity without labels is itself part of the internal–external misalignment, not a solved sub-problem—a point we return to in the discussion.

Table 12. *Predicted cluster count (mean over 5 seeds) vs. ground truth. DEAC is the deployable gate's selected engine; recovering the right granularity without labels is itself hard (part of the misalignment). Best (closest to C_{true}) in bold.*

Dataset	C_{true}	Baseline	AMR-DE	EMR-CGC	**DEAC**
Wine	3	**3**	**3**	**3**	**3**
Glass	6	**3**	10	9	9
Olivetti	40	5	13	**19**	13
CNAE-9	9	10	15	**9**	10
COIL-20	20	15	**21**	16	16
Optdigits	10	13	14	**11**	**11**
MFeat	10	12	12	**10**	**10**
Segment.	7	21	28	23	**21**
ISOLET	26	**23**	32	22	22
Satimage	6	13	29	**10**	**10**
USPS	10	11	19	**10**	11
PenDigits	10	26	34	**20**	21
MAE	—	7.53	10.57	5.25	5.77
Exact	—	1/12	1/12	4/12	2/12

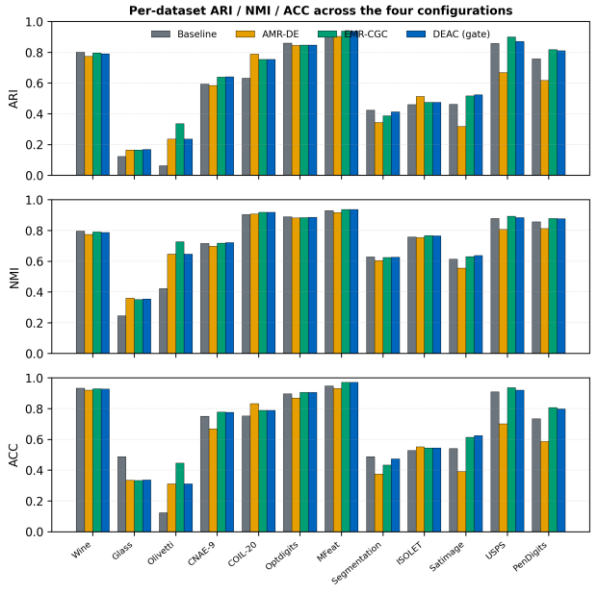


Figure 7. *Per-dataset comparison of ARI, NMI, and ACC across the four configurations (Baseline, AMR-DE, EMR-CGC, DEAC). Grouped bars make the per-dataset ranking readable; the deployable gate (DEAC) tracks the strongest engine across all three indices without dominating.*

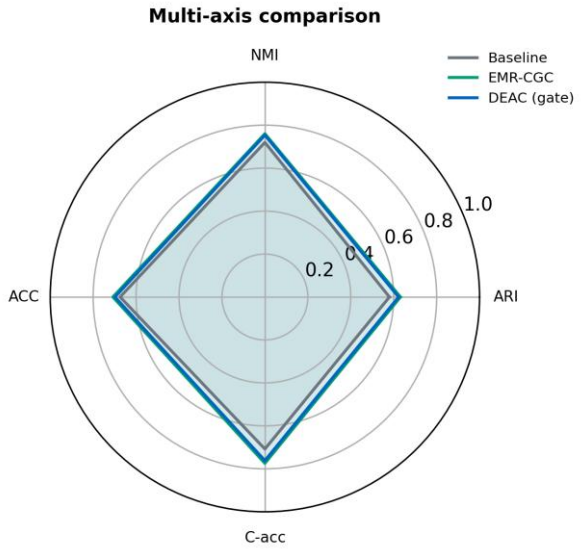


Figure 8. *Multi-axis comparison on four axes: ARI, NMI, ACC (external accuracy) and C-acc (an inverse normalised cluster-count error). The deployable gate (DEAC) tracks EMR-CGC on the external axes but is weaker on C-acc, reflecting its limited label-free cluster-count recovery.*

Statistical Significance

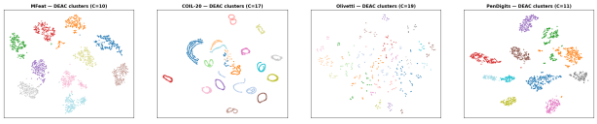
We test the *deployable* DEAC gate against the nine independent competitors—five classical (auto-C protocol of §4) and four deep—with two-tailed Wilcoxon signed-rank tests and Holm–Bonferroni correction over the family of nine (Table 13). The result is deliberately reported without euphemism: after correction, the gate significantly exceeds only Spectral and HDBSCAN ($p_{\text{Holm}} = 0.004$); against KMeans, GMM, Agglomerative, DEC, IDEC, and DCN the difference does not reach significance, and the gate trails N2D by a small margin (-0.019 mean ARI, 95% CI $[-0.079, +0.040]$; Wilcoxon $p = 0.73$, Cohen's $d_z = -0.18$). Crucially, a failure to reject is *not* equivalence: a two-one-sided-tests check does not establish equivalence to N2D even at a ± 0.05 ARI margin ($p = 0.17$), so we claim neither superiority nor equivalence—only a small, statistically unresolved difference at this sample size. The deployable method is therefore *competitive* with strong classical and deep baselines while uniquely inferring the cluster count—its value is not a higher score but the diagnostic lesson of §5. Beyond the numbers, the geometry of the resulting clusters is informative.

Table 13. *Two-tailed Wilcoxon signed-rank tests of the deployable DEAC gate vs. each comparator over 12 datasets, Holm–Bonferroni corrected across the nine tests. ✓ marks a Holm-significant DEAC advantage.*

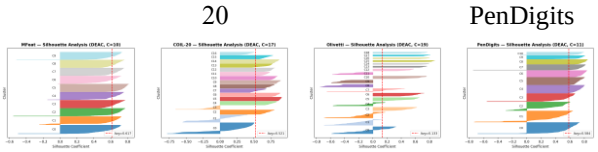
Comparison	Type	p	p_{Holm}	Sig.
DEAC vs KMeans	classical	0.034	0.205	×
DEAC vs Spectral	classical	0.000	0.004	✓
DEAC vs Agglomerative	classical	0.151	0.303	×
DEAC vs GMM	classical	0.052	0.261	×
DEAC vs HDBSCAN	classical	0.000	0.004	✓
DEAC vs N2D	deep	0.733	0.733	×
DEAC vs DEC	deep	0.064	0.261	×
DEAC vs IDEC	deep	0.064	0.261	×
DEAC vs DCN	deep	0.027	0.188	×

Qualitative Analysis

Fig. 9 shows t-SNE projections of DEAC clusters (top) and silhouette plots (bottom) on four representative datasets (on a stratified 2,000-point sample for readability). The clusters are well separated and the silhouette coefficients are predominantly positive (mean 0.62 on MFeat, 0.58 on PenDigits). The optimization that produces them is worth inspecting too.



(a) MFeat (b) COIL- (c) Olivetti (d)

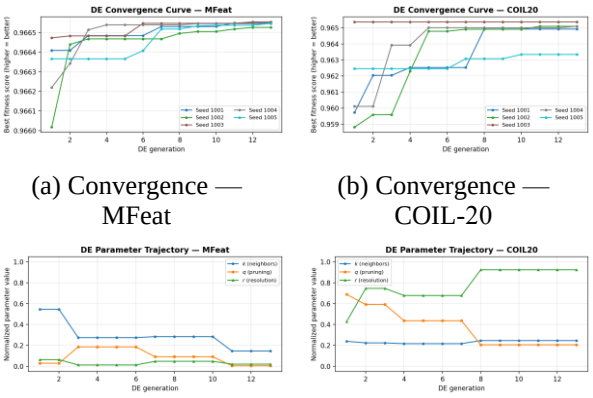


(e) MFeat sil. (f) COIL-20 sil. (g) Olivetti sil. (h) PenDigits sil.

Figure 9. *Qualitative analysis. Top (a–d): t-SNE projections of DEAC clusters. Bottom (e–h): silhouette plots.*

Optimization Dynamics

Fig. 10 shows DE convergence curves and parameter exploration trajectories. Each restart converges within 6–8 generations; the trajectory axis concatenates the $R = 2$ restarts (so it extends beyond generation 8) before Nelder–Mead refines the best candidate. The nominal evaluation budget is $R \cdot G \cdot P \approx 320$ fitness evaluations (Table 16). To separate the contribution of each component, we now ablate them.



(a) Convergence — MFeat (b) Convergence — COIL-20

(c) Trajectory — MFeat (d) Trajectory — COIL-20

Figure 10. *DE optimization dynamics: convergence curves (top) and parameter trajectories (bottom). The horizontal axis concatenates the $R = 2$ restarts, so it runs past $G = 8$ generations; Nelder–Mead then refines the best candidate.*

Ablation Study

Table 14 isolates the contribution of every DEAC component.

Table 14. *Ablation: mean ARI across 12 datasets (canonical, 5 seeds). DEAC is the deployable gate; the oracle is the label-aware upper bound.*

Configuration	Mean ARI	Δ vs. PCA
PCA Baseline (no optim.)‡	0.326	—
UMAP Baseline (no optim.)	0.580	+77.9%
UMAP + DE (AMR-DE)	0.563	+72.7%
SNN + Leiden (EMR-CGC)	0.631	+93.6%
+ QualityGate (DEAC)	**0.623**	**+91.1%**
Oracle (upper bound)	*0.646*	*+98.2%*

‡ The PCA-stage value is from a preliminary single-seed run; a controlled single-variable ablation at matched budget is left to future work.

Four effects stand out. (i) Representation matters most: moving from PCA to UMAP is the single largest improvement. (ii) The SNN–Leiden ensemble (EMR-CGC) is the workhorse, reaching 0.631—the best deployable mean ARI in the table. (iii) The optimizer AMR-DE, used alone, falls *below* the UMAP baseline (0.563 vs. 0.580)—a direct consequence of the internal–external misalignment discussed in §5. (iv) The QualityGate's label-free arbitration does not add accuracy: it reaches 0.623, *below* EMR-CGC alone, trading a small mean-ARI cost for protection against the optimizer's worst single-engine failures. The framework's accuracy therefore comes from the ensemble; the gate is a safety mechanism, and the label-aware oracle (0.646) marks the ceiling any label-free selector would need to reach. Fig. 11 stacks these increments as a waterfall.

Because the progression above changes several factors at once, we also vary each component *singly* at a matched budget (Table 15). The representation dominates—PCA→UMAP adds +0.251 mean ARI—whereas switching the community detector (Louvain→Leiden, −0.003) or the edge weighting (KNN→SNN, −0.010) changes little on average. DEAC's accuracy therefore comes from the UMAP representation and the ensemble, not from Leiden or SNN *per se*; SNN's benefit is confined to the high-dimensional datasets (§5), so the ensemble gain cannot be attributed to SNN weighting alone. Accuracy gains are only useful if the method remains affordable.

Table 15. *Controlled single-variable ablation (mean ARI, 12 datasets, 5 seeds, same budget $k = 35, r = 0.25$). Each row changes exactly one component from the row above, so the Δ isolates that factor.*

Configuration	Varied	Mean ARI (Δ)
PCA + KNN + Louvain	—	0.329
UMAP + KNN + Louvain	representation	0.580 (+0.251)
UMAP + KNN + Leiden	detector	0.577 (-0.003)
UMAP + SNN	weighting	0.567 (-0.010)

Configuration	Varied	Mean ARI (Δ)
+ Leiden		

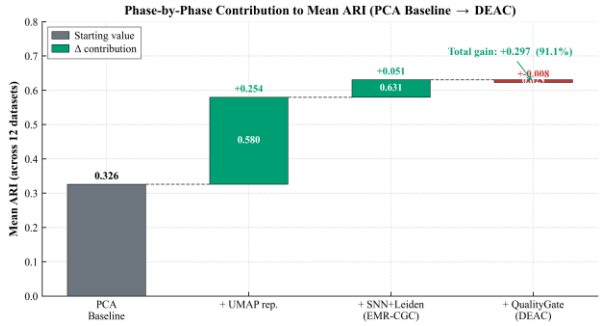


Figure 11. *Phase-by-phase contribution to mean ARI over 12 datasets. From the PCA baseline, UMAP representation and the SNN–Leiden ensemble (EMR-CGC) drive mean ARI to 0.631; the label-free QualityGate then reduces it slightly to 0.623 (trading accuracy for safety), while the label-aware oracle marks the 0.646 upper bound.*

Computational Complexity

Table 16 summarizes the theoretical cost of each component.

Table 16. *Computational complexity of DEAC components.*

Component	Complexity	Typical Cost
StandardScaler	$\mathcal{O}(nd)$	linear
PCA	$\mathcal{O}(nd \cdot d')$	$\mathcal{O}(nd \cdot 40)$
UMAP	$\mathcal{O}(n^{1.14}d')$	near-linear
t-SNE (Barnes–Hut)	$\mathcal{O}(n\log n)$	near-linear*
KNN graph	$\mathcal{O}(nk \cdot d')$	$\mathcal{O}(nk \cdot 40)$
SNN graph	$\mathcal{O}(nk^2)$	$\mathcal{O}(n \cdot k^2)$
Leiden	$\mathcal{O}(m\log n)$	near-linear
AMR-DE (all evals)	$\mathcal{O}(E \cdot F_c)$	$E = RGP \approx 320$
EMR-CGC ensemble	$\mathcal{O}(M\|K\|\|R\|F_c)$	$60 \cdot F_c$
Total DEAC	$\mathcal{O}(n^{1.14}d' + nk^2)$	sub-quadratic

* t-SNE uses the Barnes–Hut approximation (output dimension ≤ 3), which is $\mathcal{O}(n\log n)$; exact t-SNE would be $\mathcal{O}(n^2)$. The sub-quadratic total therefore holds for the implemented pipeline, with the $\mathcal{O}(nk^2)$ SNN term dominating.

Measured end-to-end wall-clock for the full three-engine pipeline (single seed, one machine) is 29 s on Wine ($n = 178$), 102 s on MFeat ($n = 2000$), and 200 s on USPS ($n = 9298$), with peak memory rising from 0.6 to 1.8 GB. EMR-CGC's 60-run ensemble and the AMR-DE search dominate the cost, so runtime grows with n but stays within a few minutes at the high end of our benchmarks.

5. Discussion

The results raise four follow-up questions: why arbitration rather than optimization, how much of the oracle a label-free gate can recover, why SNN rather than KNN, and where the method still falls short.

Why Arbitration, Not Optimization?

The most informative result in our study is a negative one. AMR-DE explicitly maximizes internal objectives (modularity, silhouette, Calinski–Harabasz), yet its mean ARI (0.563) falls *below* the fixed baseline (0.580) and it collapses on several datasets (Satimage 0.318, Segmentation 0.344, PenDigits 0.617). Internal quality and external agreement are therefore misaligned: a partition can be "better" by every internal score and worse by ARI. This is why DEAC does not trust any single optimized candidate; the QualityGate instead *arbitrates*, keeping an engine only when it survives three conservative, label-free tests and otherwise falling back. Crucially, the *same* misalignment that sinks AMR-DE also limits the gate: deciding by internal metrics, it reaches 0.623—*below* the ensemble it arbitrates (EMR-CGC, 0.631). Arbitration is thus a *defensive* mechanism that curbs the optimizer's worst losses, not an accuracy-maximizing one. A controlled selection-rule ablation (Table 17) confirms this: removing the over-fragmentation guard leaves the gate essentially unchanged, dropping AMR-DE from the gate costs only ~ 0.003 mean ARI, and the trivial "always-EMR-CGC" policy (0.631) in fact exceeds the gate. The gate therefore buys robustness against catastrophic single-engine picks rather than higher mean accuracy. How much of the label-aware oracle any such label-free rule can recover is the central question.

Table 17. *Deployable selection-rule ablation (mean ARI, 12 datasets, 5 seeds). Removing the rho guard (best-internal) and removing AMR-DE from the gate both leave performance essentially unchanged; the trivial 'always-EMR-CGC' policy slightly exceeds the gate. The gate's role is protective (it avoids the optimizer's worst picks), not accuracy-maximizing.*

Deployable policy	Mean ARI	Oracle-gain recovery
Always-Baseline (no selection)	0.580	+0%
Always-EMR-CGC	0.631	+77%
Best-internal (no rho guard)	0.624	+66%
QualityGate w/o AMR-DE	0.621	+62%
QualityGate (full)	0.624	+66%
Oracle (upper	0.646	+100%

Deployable policy	Mean ARI	Oracle-gain recovery

bound)

How Much of the Oracle Is Deployable?

The main results (Table 5) select the best engine per dataset using ground-truth ARI—an oracle unavailable at deployment, where the gate must choose from internal metrics alone. Table 18 quantifies the gap (five seeds, full data): the deployable gate reaches mean ARI 0.623 against the oracle's 0.646 and the baseline's 0.580, recovering 65% of the oracle's gain over the baseline and agreeing with the oracle's engine choice on 6 of 12 datasets. The residual gap is a direct expression of the internal–external misalignment that motivates this work: the gate errs exactly where an engine is best by ARI but not by internal score—most visibly on Olivetti, where it prefers AMR-DE (0.237) over the stronger EMR-CGC (0.337). The same mechanism makes the gate non-monotone over the baseline—on Wine it prefers AMR-DE and loses 0.011 ARI—and, in aggregate, leaves it slightly below EMR-CGC alone (0.623 vs 0.631). We therefore treat the oracle as an upper bound and present the gate as a conservative, honestly deployable rule, and regard closing their gap—better label-free proxies for external quality—as the central open problem. The ensemble's own ingredients invite scrutiny, beginning with the SNN graph on which EMR-CGC is built.

Table 18. *Deployable QualityGate (internal metrics, no labels) vs. the oracle upper bound, mean ARI over 5 seeds.* Δ *= oracle* $-$ *gate.*

Dataset	Gate ARI	Oracle ARI	Δ
Wine	0.791	0.802	+0.011
Glass	0.169	0.172	+0.003
Olivetti	0.237	0.337	+0.101
CNAE-9	0.642	0.642	+0.000
COIL-20	0.754	0.789	+0.035
Optdigits	0.848	0.861	+0.013
MFeat	0.939	0.939	+0.000
Segment.	0.413	0.437	+0.024
ISOLET	0.476	0.516	+0.040
Satimage	0.525	0.541	+0.016
USPS	0.871	0.900	+0.029
PenDigits	0.811	0.819	+0.008
Mean	**0.623**	0.646	+0.023

SNN vs. KNN Graph Construction

SNN weighting is not a uniform win, and it is worth being precise about where it helps. Under a matched single-graph protocol (same UMAP representation, $k = 35$, $r = 0.25$), swapping KNN+Louvain for SNN+Leiden changes mean ARI only marginally—from 0.580 to 0.568, a 1.2-point decrease on average. The average, however, hides a clear pattern that follows the theory of §3.6: SNN improves precisely on the *high-dimensional* datasets, where distance concentration degrades raw-distance weights—Olivetti ($d = 4096$, $+0.027$), CNAE-9 ($d = 856$, $+0.028$), and COIL-20 ($d = 1024$, $+0.038$)—while it costs accuracy on lower-dimensional inputs (USPS -0.076, PenDigits -0.058). SNN is therefore a targeted tool for high-dimensional regimes, not a blanket improvement, and EMR-CGC's overall gain comes primarily from *ensemble diversity* (five representations × twelve configurations with best-run selection) rather than from the SNN weighting alone. Neither choice removes every limitation, which we list next.

Robustness

To probe where the method holds and breaks, we stress it on controlled synthetic data (`make_classification`, ground truth known) along four axes (Table 19). The gate is *robust to class imbalance* (mean ARI 0.49–0.60 from balanced up to a 30 × ratio) and *improves with larger* n (0.32 at $n = 1000$ to 0.79 at $n = 10000$) and with cluster separation (up to 0.97). Its clear failure mode is *irrelevant dimensions*: adding noise features to the 20 informative ones collapses ARI from 0.52 to near zero—and it is *every* engine that fails, not just the gate, because the UMAP graph is built on a distance that uninformative features dominate. DEAC therefore carries a scope condition: it assumes the informative signal is not swamped by irrelevant features.

Table 19. *Robustness of the deployable gate on controlled synthetic data (mean ARI over 3 seeds; 'base' is* $n = 2000$, $d = 20$ *all-informative,* $C = 10$*, balanced, class-sep 1.0). The gate is robust to class imbalance, improves with cluster separation and with larger* n*, but degrades sharply under many irrelevant dimensions—a limitation noted in §5.*

Stress axis	Level	Gate ARI	Baseline ARI
Imbalance	base	0.515	0.413
	3×	0.487	0.450
	10×	0.605	0.605
	30×	0.599	0.631
Separation	0.5	0.059	0.059
	0.75	0.222	0.178
	base	0.515	0.413
	1.5	0.849	0.830
	2.0	0.966	0.966
Irrelevant dims	base	0.515	0.413
	+50	0.017	0.017
	+200	0.002	0.002
	+500	0.000	0.000
Sample	1000	0.324	0.229

Stress axis	Level	Gate ARI	Baseline ARI
size n			
	base	0.515	0.413
	5000	0.649	0.595
	10000	0.787	0.775

Limitations

(i) *Runtime.* EMR-CGC's 60-run ensemble is costlier than single-shot methods—seconds to a few minutes per dataset in our experiments—and is the dominant cost of the pipeline. (ii) *Internal–external gap.* The gate decides on internal metrics (MOD, SIL, BAL), which do not align with ARI, so it is not strictly monotone over the baseline—on Wine it prefers AMR-DE and loses 0.011 ARI—and in aggregate scores below the EMR-CGC ensemble it arbitrates (0.623 vs 0.631); its deployable cluster-count error is also large (Table 12). (iii) *Scalability.* SNN is $\mathcal{O}(nk^2)$, so very large n ($>$ 50,000) would require approximate-NN construction. (iv) *Oracle vs. deployable gap.* The label-free gate recovers 65% of the oracle's gain over the baseline and trails N2D (Table 18). Across learned, stability, and consensus selectors the gap does not close (best 64%), so closing it—better label-free proxies for external quality, or a learned notion of it—is the primary open problem and plausibly requires some supervision. (v) *Irrelevant features.* Like other distance/graph methods, DEAC degrades sharply when informative features are outnumbered by irrelevant ones (Table 19); it presumes a modest feature-relevance ratio or a prior feature-selection step.

6. Conclusion

We presented evidence that internal clustering metrics are an unreliable proxy for external accuracy—an engine that maximizes modularity and silhouette scores *below* an unoptimized baseline (0.563 vs 0.580 mean ARI)—and a characterization of how far a label-free rule can close the resulting oracle–deployable gap. The framework, DEAC (Defensive Ensemble Adaptive Clustering), makes this precise: a multi-representation SNN–Leiden ensemble (EMR-CGC) that infers the cluster count and is competitive with strong classical and deep baselines (0.631 mean ARI, trailing only N2D), arbitrated by a three-layer QualityGate that acts as a conservative safety net (0.623) rather than an accuracy booster. The label-aware oracle (0.646) only matches N2D, and no label-free selector we tested—metric re-weighting, learned, stability, or consensus—recovers more than 65% of its gain over the baseline, indicating the gap is structural. The lesson is that the bottleneck in automatic clustering is not clustering power but the inability to *recognize* a good partition without labels. Three extensions remain open.

The first is approximate-NN SNN construction, which would extend DEAC beyond $n = 50$k. The second is integrating deep representation learning into the RepScan stage. The third is a semi-supervised variant that uses partial labels to close the internal–external metric gap identified in §5.

Code and data availability. The framework, all experiment scripts, and the oracle–deployable gap study are available at https://github.com/al-shaibani-star/-DEAC.

A. K. Jain, "Data clustering: 50 years beyond K-means," *Pattern Recognition Letters*, vol. 31, no. 8, pp. 573–580, 2010.

M. Steinbach, L. Ertöz, and V. Kumar, "The challenges of clustering high dimensional data," in *New Directions in Statistical Physics*, Springer, 2004, pp. 273–309.

S. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, 1982.

A. Y. Ng, M. I. Jordan, and Y. Weiss, "On spectral clustering: Analysis and an algorithm," in *Proc. NeurIPS*, 2001, pp. 849–856.

R. J. G. B. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Proc. PAKDD*, Springer, 2013, pp. 160–172.

S. E. Schaeffer, "Graph clustering," *Computer Science Review*, vol. 1, no. 1, pp. 27–64, 2007.

V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Stat. Mech.*, vol. 2008, no. 10, p. P10008, 2008.

V. A. Traag, L. Waltman, and N. J. van Eck, "From Louvain to Leiden: guaranteeing well-connected communities," *Scientific Reports*, vol. 9, no. 1, p. 5233, 2019.

I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. Springer, 2002.

L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform Manifold Approximation and Projection for dimension reduction," *arXiv:1802.03426*, 2018.

L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, pp. 2579–2605, 2008.

R. Storn and K. Price, "Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces," *J. Global Optim.*, vol. 11, no. 4, pp. 341–359, 1997.

S. Mirjalili, S. M. Mirjalili, and A. Lewis, “Grey Wolf Optimizer,” *Advances in Engineering Software*, vol. 69, pp. 46–61, 2014.

X.-S. Yang, *Nature-Inspired Metaheuristic Algorithms*, 2nd ed. Luniver Press, 2010.

H. Faris, I. Aljarah, M. A. Al-Betar, and S. Mirjalili, “Grey wolf optimizer: a review of recent variants and applications,” *Neural Computing and Applications*, vol. 30, no. 2, pp. 413–435, 2018.

L. Ertöz, M. Steinbach, and V. Kumar, “Finding clusters of different sizes, shapes, and densities in noisy, high dimensional data,” in *Proc. SIAM Int. Conf. Data Mining*, 2003, pp. 47–58.

Y. Wang and M. J. Samonte, “Density peak clustering algorithm based on adaptive K-nearest neighbors for manifold data,” *Discover Applied Sciences*, vol. 8, art. 165, 2026.

A. Strehl and J. Ghosh, “Cluster ensembles—a knowledge reuse framework for combining multiple partitions,” *J. Mach. Learn. Res.*, vol. 3, pp. 583–617, 2002.

A. L. N. Fred and A. K. Jain, “Combining multiple clusterings using evidence accumulation,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 27, no. 6, pp. 835–850, 2005.

S. Vega-Pons and J. Ruiz-Shulcloper, “A survey of clustering ensemble algorithms,” *Int. J. Pattern Recognit. Artif. Intell.*, vol. 25, no. 3, pp. 337–372, 2011.

L. Hubert and P. Arabie, “Comparing partitions,” *J. Classification*, vol. 2, no. 1, pp. 193–218, 1985.

Y. Yang, D. Xu, F. Nie, S. Yan, and Y. Zhuang, “Image clustering using local discriminant models and global integration,” *IEEE Trans. Image Process.*, vol. 19, no. 10, pp. 2761–2773, 2010.

S. Das, A. Abraham, and A. Konar, “Automatic clustering using an improved differential evolution algorithm,” *IEEE Trans. Syst., Man, Cybern. A*, vol. 38, no. 1, pp. 218–237, 2008.

E. Becht *et al.*, “Dimensionality reduction for visualizing single-cell data using UMAP,” *Nature Biotechnology*, vol. 37, pp. 38–44, 2019.

M. R. Brito, E. L. Chávez, A. J. Quiroz, and J. E. Yukich, “Connectivity of the mutual k-nearest-neighbor graph in clustering and outlier detection,” *Statistics & Probability Letters*, vol. 35, no. 1, pp. 33–42, 1997.

V. Satuluri, S. Parthasarathy, and D. Ruan, “Local graph sparsification for scalable clustering,” in *Proc. ACM SIGMOD*, 2011, pp. 721–732.

J. Xie, R. Girshick, and A. Farhadi, “Unsupervised deep embedding for clustering analysis,” in *Proc. ICML*, 2016, pp. 478–487.

R. McConville, R. Santos-Rodriguez, R. J. Piechocki, and I. Craddock, “N2D: (Not too) deep clustering via clustering the local manifold of an autoencoded embedding,” in *Proc. ICPR*, 2021, pp. 5145–5152.

X. Guo, L. Gao, X. Liu, and J. Yin, “Improved deep embedded clustering with local structure preservation,” in *Proc. IJCAI*, 2017, pp. 1753–1759.

B. Yang, X. Fu, N. D. Sidiropoulos, and M. Hong, “Towards K-means-friendly spaces: Simultaneous deep learning and clustering,” in *Proc. ICML*, 2017, pp. 3861–3870.

K. V. Mardia, J. T. Kent, and J. M. Bibby, *Multivariate Analysis*. Academic Press, 1979.

K. Beyer, J. Goldstein, R. Ramakrishnan, and U. Shaft, “When is “nearest neighbor” meaningful?” in *Proc. ICDT*, 1999, pp. 217–235.

M. E. Houle, H.-P. Kriegel, P. Kröger, E. Schubert, and A. Zimek, “Can shared-neighbor distances defeat the curse of dimensionality?” in *Proc. SSDBM*, 2010, pp. 482–500.

A. Ghriss and C. Monteleoni, “Deep clustering via probabilistic ratio-cut optimization,” in *Proc. AISTATS*, 2025.